Character Handling functions in C

C - Character Functions

Library Functions - Input Character

Functions to input a single character are available in 'stdio.h' header file. These functions are:

- **getch()**: 'get' stands for input, 'ch' stands for character. Function getch() is use to input single character, but it will not display the character entered by the user.
  Syntax: char a = getch();
- **getche()**: getche() is same as getch() function only the difference is that it will display the character entered by the user. 'get' stands for input, 'ch' stands for character, and 'e' stands for echo(display)
  Syntax: char a = getche();
  Both getch() and getche() functions will not require enter key to terminate.
- **getchar()**: It will terminate by enter key after inputting a single character.
  Syntax: char a = getchar();

C - Character Functions

C Character functions are used to identify whether the input character is alphabet, digit, lowercase, uppercase etc.

Built-in C - Character functions in ctype.h are:

S.No	Character Function	Description
1.	isalpha()	Return true if character is an alphabet or not.
2.	isdigit()	Return true if character is a digit or not.
3.	isalnum()	Return true if character is an alphabet, number or not.
4.	islower()	Return true if character is in lowercase or not.
5.	isupper()	Return true if character is in uppercase or not.
6.	isspace()	Return true if character is a space or not.
7.	ispunct()	Return true if character is punctuation or not.

C - Character Functions

Input Character and display it.

```
#include<stdio.h>
void main()
{
  char a;
  printf("Enter character: ");
  a = getch();
  //a = getche();
  //a = getchar();
  printf("\nYou have pressed: %c",a);
  //putch(a);
  //putchar(a);
  //Output of putch and putchar is same
getch();
}
```

C - Character Functions

Program to explain difference between getch() and getche() function.

```
void main()
{
  char ch;
  printf("Enter character: ");
  ch=getch();
  printf("\nch=%c",ch);
  printf("\nEnter character: ");
  ch=getche();
  printf("\nch=%c",ch);
}
```

Output:

```
Enter character:
ch=a
Enter character: a
ch=a
```

C - Character Functions

WAP to check whether the given character is Alpha-numeric or not.

```
void main()
{
  char ch;
  printf(“Enter character: ");
  ch=getche();
  if(isalnum(ch))
        printf("\nIt is alpha-numeric character.");
  else
        printf("\nIt is not an alpha-numeric character.");
}
```

C - Character Functions

WAP to check whether the given character is Alphabet or not.

```
void main()
{
  char ch;
  printf("Enter character: ");
  ch=getche();
  if(isalpha(ch))
          printf("\nIt is a alphabet character.");
  else
           printf("\nIt is not an alphabet character.");
}
```

C - Character Functions

Exercise

- WAP to check input character is a vowel or not.
- WAP to input a character and find it is in uppercase, or in lowercase, or a digit or a special symbol using character functions.
- WAP to input a character and find it is in uppercase, or in lowercase, or a digit or a special symbol without using character functions.
- Input character in lowercase and convert into uppercase. [result = toupper(c);]
- Input character in uppercase and convert into lowercase.[result = tolower(c);]

Arithmetic Operations on Characters

- Character range is between -128 to 127 or 0 to 255.
- This point has to be kept in mind while doing character arithmetic.
- Character arithmetic is used to implement arithmetic operations like addition, subtraction, multiplication, and division on characters in C language.
- In character arithmetic character converts into an integer value to perform the task.
- For this ASCII value is used. It is used to perform actions on the strings.

Arithmetic Operations on Characters

- %d specifier causes an integer value to be printed and
- %c specifier causes a character value to printed.

```
int main(void)
{
    char value1 = 'a';//a=97
    char value2 = 'b';//b=98
    char value3 = 'z';//z=122
    // perform character arithmetic
    char num1 = value1 + 3;
    char num2 = value2 - 1;
    char num3 = value3 + 2;
    // print value
    printf("numerical value=%d\n", num1);
    printf("numerical value=%d\n", num2);
    printf("numerical value=%d\n", num3);
    return 0;
}
```

```
Output
numerical value=100
numerical value=97
numerical value=124
```

Arithmetic Operations on Characters

```
int main()
{
    char a = 'A';
    char b = 'B';

    printf("a = %c\n", a);
    printf("b = %c\n", b);
    printf("a + b = %c\n", a + b);

    return 0;
}
```

```
Output
a = A
b = B
a + b = â
```

Explanation

In this program, two character variables a and b are declared and assigned the values ‘A’ and ‘B’, respectively. The program then adds a and b using character arithmetic, which results ‘â’. The result is then printed using the printf() function.

Note that in character arithmetic, the characters are treated as integers based on their ASCII code values. For example, the ASCII code for ‘A’ is 65 and for ‘B’ is 66, so adding ‘A’ and ‘B’ results in 65 + 66 = 131, which is the ASCII code for ‘â’.

Character Input

- To read a single character as input we use the getchar() function.
- We will take a character as input from the user and will print the ASCII code of the character.

```
#include <stdio.h>
int main()
{
  char ch;
  printf("Enter any character: ");
  ch = getchar();
  printf("Entered character: %c\n", ch);
  printf("ASCII value: %d\n", ch);
  return 0;
}
```

Output

```
Enter any character: A
Entered character: A
ASCII value: 65
```

Character Output

- To output a single character we use the putchar() function.
- In the following example we take a single character as input and then output it using putchar() function.

```
#include <stdio.h>
int main()
{
  char ch;
  printf("Enter any character: ");
  ch = getchar();
  printf("Entered character: ");
  putchar(ch);
  return 0;
}
```

Output

```
Enter any character: A
Entered character: A
```

gcc xyz.c -o xyz -lm

- **gcc** is compiler command (compiler name)
  xyz.c is a source file name.
  -o is an option to specify the output file.
  xyz is the name of the output file.
  -lm is an option to link againt the math library (*libm*).